# VAANISHIELD: CODE-SWITCH-AWARE DEEPFAKE VOICE DETECTION FOR FINANCIAL FRAUD CALLS IN INDIAN MULTILINGUAL SPEECH

Advanced Artificial Intelligence (CSE_3139) — V Semester Mini Project

**COMPREHENSIVE TEAM PRESENTATION SCRIPT DECK & CODE REPOSITORY MAPPING**

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
*(A constituent unit of MAHE, Manipal)*

**MANIPAL ACADEMY OF HIGHER EDUCATION**

School of Computer Engineering, Manipal Institute of Technology, Bengaluru

B.Tech Computer Science & Engineering (Artificial Intelligence)


| Role & Member | Registration No. | Specialization & Core Architecture Domain |
|---|---|---|
| **Dhruv Gourisaria** (Team Lead & System Architect) | **245890486** | Overall System Architecture, Dual-Stream Hierarchical Fusion, ARS-SNAP Regularized Projection, ONNX Engine, Low-Latency Optimization, API Orchestration |
| **Jessica Mariam Mathews** (Acoustic Lead) | **245890380** | Acoustic Feature Extraction, Code-Switch Boundary Localization, Vectorized YIN Pitch Tracking, Energy Jump Dynamics, Monolingual Fallback Routing |
| **Samarth Bangwal** (Evaluation Lead) | **245890410** | Enterprise Corpus Construction (1,600 utts), Commercial Out-of-Domain Synthesizer Evaluation (Edge-TTS), Telephony Codec Stress Tests, Demographic Gender Parity Analysis |
| **Aadi Harnad Bhat** (Deployment Lead) | **245890054** | Enterprise Banking Tier Calibration, SHA-256 Audit Certification Engine, Full-Duplex Streaming WebSocket Server, React Vite Security Dashboard |

Under the guidance of:

**Dr. Mahadevi S.**

Assistant Professor, School of Computer Engineering

Manipal Institute of Technology, Bengaluru



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
*(A constituent unit of MAHE, Manipal)*


**SCHOOL OF COMPUTER ENGINEERING — OCTOBER 2026**

# 1. EXECUTIVE OVERVIEW & PRESENTATION TIMELINE

This document establishes the official spoken script, presentation flow, and exact source-code mapping for the academic evaluation of **VaaniShield**. The presentation is allocated a total of **20 minutes** (16 minutes presentation + 4 minutes faculty Q&A; / live system defense), rigorously distributed among all four team members to demonstrate balanced technical ownership, depth of research, and enterprise-grade deployment readiness.

| Part | Presenter & Role | Topic & Architecture Domain | Time | Key Source Files |
|---|---|---|---|---|
| **Part I** | **Dhruv Gourisaria** (Team Lead) | Problem Statement, Architectural Innovation, Dual-Stream Fusion & ARS-SNAP | 4.0 min | classifier.py<br>snap_projection.py<br>onnx_engine.py |
| **Part II** | **Jessica Mariam Mathews** (Acoustic Lead) | Micro-Acoustic Seams, Vectorized YIN Pitch, Monolingual Fallback Routing | 4.0 min | features.py<br>localization.py<br>audio_io.py |
| **Part III** | **Samarth Bangwal** (Evaluation Lead) | Enterprise Multilingual Corpus, Out-of-Domain Synthesis, Telephony Stress Tests | 4.0 min | build_scaled_corpus.py<br>metrics.py<br>ablation.py |
| **Part IV** | **Aadi Harnad Bhat** (Deployment Lead) | Banking Tiers, SHA-256 Audit Engine, WebSocket Pipeline & React Dashboard | 4.0 min | certificate.py<br>server.py<br>App.jsx |
| **Q&A;** | **All Team Members** | Panel Cross-Examination, Live Demonstration & Code Inspection | 4.0 min | Full Repository |

# 2. PART I: DHRUV GOURISARIA — ARCHITECTURAL INNOVATION & MULTI-STREAM DUAL PROJECTION

**Presenter: Dhruv Gourisaria (Reg. No: 245890486) | Role: Team Lead & System Architect**
**Presentation Window: 00:00 – 04:00 (4 Minutes) | Target Slides: Slides 1–4**

## A. Slide-by-Slide Detailed Spoken Script

### Slide 1: Title & The Telephonic Voice Cloning Crisis in Indian Banking (00:00 - 01:00)

*"Good morning Respected Evaluators and Dr. Mahadevi. I am Dhruv Gourisaria, lead researcher on project VaaniShield. Over the past eighteen months, Indian financial institutions have faced an unprecedented threat: telephonic voice cloning fraud. High-fidelity neural voice cloners allow attackers to clone an executive or family member's voice using as little as three seconds of publicly scraped audio, executing fraudulent wire transfers and simulated kidnapping ransoms over standard phone channels.*

*When we examined state-of-the-art anti-spoofing models—such as AASIST, RawNet2, and LCNN—we discovered an alarming vulnerability: these models were benchmarked on Western datasets like ASVspoof 2019 and 2021, which feature controlled, monolingual English in pristine studio environments. Real-world Indian telephone calls, however, are intrinsically multilingual and code-switched—fluidly alternating between English and Indic languages like Hindi, Tamil, Telugu, and Kannada within a single sentence. Confronted with code-switching, conventional classifiers suffer catastrophic failure: they mistake natural phonetic articulatory resets for deepfake manipulation, causing false alarm rates to surge beyond 40%, while overfitting to the high-frequency spectral artifacts of their training vocoders. VaaniShield was engineered specifically to solve this structural vulnerability."*

### Slide 2: The Core Technological Breakthrough — Dual-Stream Architecture (01:00 - 02:00)

*"To protect Indian multilingual banking telephony, VaaniShield abandons monolithic black-box classification in favor of a hierarchical, dual-stream acoustic architecture. Our insight is grounded in the physical mechanics of neural speech synthesis: commercial voice cloners concatenate or synthesize code-switched phrases through neural vocoders that fail to preserve micro-prosodic continuity across language transitions.*

*VaaniShield operates two parallel detection streams: First, an ultra-low-latency Boundary-Local Prosodic & Phase Seam Stream that zooms in on detected code-switching transition points, capturing pitch acceleration, short-time energy steps, spectral flux variance, and phase coherence resets. Second, a global Semantic-Acoustic Representation Stream powered by self-supervised speech foundations—specifically WavLM and XLSR. However, feeding raw foundation embeddings directly into a classifier introduces severe vocoder overfitting and identity bias. To overcome this, we invented ARS-SNAP: Adaptive Regularized Subspace Projection."*

### Slide 3: Mathematical Formalism of ARS-SNAP (02:00 - 03:00)

*"Let me explain the mathematical mechanics of ARS-SNAP, implemented in our speaker_null module. Standard SNAP projects foundation embeddings onto the orthogonal null space of speaker identity via a rigid projection matrix: P_null = I - V V^T. While this eliminates speaker identity, rigid orthogonal projection discards subtle synthetic artifacts and suffers from numerical rank deficiency when applied to small enrollment pools.*

*Under ARS-SNAP, we formulate an adaptive regularization objective with a ridge penalty and soft-nulling parameter: P_ARS = I - sum_k alpha_k (v_k v_k^T) / (lambda_k + epsilon), where v_k represents the principal eigenvectors of the speaker covariance matrix, and alpha_k dynamically scales between 0.70 and 0.95 based on subspace energy ratio. This soft-nulls the dominant speaker-identity subspace while preserving boundary-adjacent acoustic cues. Empirically, ARS-SNAP achieves an Equal Error Rate of 20.62% on unseen commercial speech synthesizers like Microsoft Edge Neural TTS—a 5.7% relative improvement over hard SNAP and over 22% improvement over raw foundation embeddings."*

### Slide 4: Zero-Pickle Security & Production ONNX Inference Engine (03:00 - 04:00)

*"Finally, as system architect, I enforced strict zero-pickle deserialization security and latency guarantees. In enterprise banking environments, standard Python pickle checkpoints represent an intolerable Remote Code Execution vulnerability (CWE-502). We developed an automated export pipeline that serializes our dual-stream fusion classifier into a self-contained, statically typed ONNX runtime engine with SHA-256 integrity verification.*

*Using ONNX Runtime with pure-NumPy SIMD operators, we achieve an end-to-end classification inference latency of 1.82 ms, bringing total pipeline latency to 127.31 ms—well beneath the 300 ms strict telephony deadline for live calls. I will now hand over to Jessica Mariam Mathews, who led our boundary acoustic and pitch tracking engineering."*

## B. Dedicated Source Code Files & Line-by-Line Technical Analysis

### 1. `vaanishield/fusion/classifier.py (Lines 1–650) — Dual-Stream Fusion Classifier`

- Implements the core VaaniShieldClassifier, managing the concatenation and multi-stage classification of the 14-dimensional boundary acoustic seam vector with the 128-dimensional ARS-SNAP projected foundation embedding.
- Implements temperature-scaled sigmoid calibration and calibrated posterior probability calculation with Neyman-Pearson thresholding (Line 245–310), ensuring banking-grade False Acceptance Rate guarantees.
- Houses the pure-NumPy inference fallback path that runs seamlessly in memory-constrained edge environments without requiring PyTorch GPU runtimes.

### 2. `vaanishield/speaker_null/snap_projection.py (Lines 1–221) — ARS-SNAP Projection Engine`

- Computes the empirical speaker covariance matrix across enrolled speakers and executes Singular Value Decomposition with eigenvalue shrinkage (Line 54–112) to derive the speaker-identity subspace.
- Applies the regularized soft-nulling equation P_ARS = I - V @ diag(alpha / (sigma + eps)) @ V.T, preventing projection singularity and preserving out-of-domain synthetic detection cues.
- Validates numerical stability via condition number checks and unit-norm projection invariance tests.

### 3. `vaanishield/fusion/onnx_engine.py (Lines 1–167) — Zero-Pickle ONNX Scoring Runtime`

- Implements the enterprise VaaniShieldONNXEngine which loads statically compiled .onnx graphs using ONNX Runtime CPU/CUDA Execution Providers (Line 42–88).
- Enforces strict cryptographic integrity: reads the model file bytes, computes its SHA-256 hash, and compares it against a signed manifest before loading into memory, preventing model poisoning attacks.
- Eliminates the Python GIL during batched inference, enabling multi-threaded microservice execution under heavy telephonic load.

### 4. `scripts/export_onnx.py (Lines 1–142) — Production ONNX Model Compilation`

- Handles the automated export of trained PyTorch fusion models to ONNX Opset 17 with dynamic batch and sequence axes.
- Validates numerical parity between PyTorch float32 output tensors and ONNX Runtime evaluation across 1,000 synthetic test samples, verifying maximum absolute error < 1e-5.

# 3. PART II: JESSICA MARIAM MATHEWS — MICRO-ACOUSTIC SEAM ANALYSIS & VECTORIZED PITCH DYNAMICS

**Presenter: Jessica Mariam Mathews (Reg. No: 245890380) | Role: Acoustic Engineering Lead**
**Presentation Window: 04:00 – 08:00 (4 Minutes) | Target Slides: Slides 5–8**

### A. Slide-by-Slide Detailed Spoken Script

#### Slide 5: Micro-Acoustic Seams at Code-Switching Boundaries (04:00 - 05:00)

*"Thank you, Dhruv. Good morning everyone. I am Jessica Mariam Mathews, and my focus was the physical acoustics of code-switching speech. In Indian bilingual conversations—such as Hindi–English or Tamil–English—human speakers transition seamlessly between phonological systems: vocal fold tension, tongue position, and formant trajectories follow continuous biological constraints.*

*However, when generative voice cloning systems synthesize code-switched utterances, they frequently rely on neural vocoders or multi-stage acoustic models that generate phrases in linguistic chunks. At the junction where the speaker switches languages, the neural synthesis pipeline exhibits micro-acoustic stitching artifacts: abrupt pitch steps (ΔF0), unnatural root-mean-square energy jumps (ΔE), spectral flux discontinuities, and phase incoherence resets across consecutive STFT frames. In our research, we isolated a 200 ms bilateral analysis window centered exactly on the code-switch transition point to capture these tell-tale mechanical seams."*

#### Slide 6: Boundary Localization & Vectorized YIN Algorithm (05:00 - 06:00)

*"Accurate boundary seam analysis requires two critical engineering components: First, temporal localization of the code-switch point. In our localization module, we utilize an energy-normalized spectral cosine distance metric evaluated across 40 ms sliding windows. By tracking localized peaks in phonological divergence, our boundary localizer identifies language switch points with a median temporal error of less than 32 ms.*

*Second, pitch tracking at these boundaries historically represented our greatest latency bottleneck. Standard Praat or Librosa YIN implementations rely on sequential time-domain difference functions with O(N^2) complexity, taking over 450 ms per 3-second call. To solve this, I implemented a pure-NumPy vectorized YIN pitch tracker that reformulates the cumulative mean normalized difference function via FFT-based autocorrelation with O(N log N) complexity. This compressed our pitch extraction latency from 450 ms to just 34.12 ms—a 13x speedup—while completely eliminating external C-library dependencies."*

### Slide 7: 14-Dimensional Boundary Acoustic Vector Formulation (06:00 - 07:00)

*"Across each detected transition window, our boundary feature extraction engine computes a compact, 14-dimensional feature vector specifically engineered to expose synthetic seams:*

*1. Max Pitch Jump (ΔF0): Measures vocal cord fundamental frequency acceleration across the boundary.*

*2. Mean Pitch Velocity: Captures anomalous glottal slew rates exceeding human physiological limits.*

*3. Voicing Ratio Delta: Detects abrupt, synthetic transitions between voiced and unvoiced frames.*

*4. Energy Jump Ratio (ΔE): Computes short-time log-RMS energy steps across the pre- and post-boundary segments.*

*5. Spectral Flux Jump: Captures sudden spectral envelope discontinuities across consecutive STFT frames.*

*6. Phase Reset Coherence: Measures instantaneous phase divergence across harmonic frequency bins.*

*7. High-Frequency Spectral Roll-off Discontinuity: Detects vocoder band-limiting and neural upsampling boundaries.*

*Because this 14-D vector is anchored in the physical laws of human vocal tract physiology, it generalizes remarkably well across completely unseen generative vocoders."*

### Slide 8: Dual-Mode Monolingual Fallback Routing (07:00 - 08:00)

*"A critical real-world challenge was: what happens when a customer makes a purely monolingual phone call without any code-switching? Under a naive boundary-detection model, the boundary feature vector would collapse to all zeros, blinding the classifier or triggering false alarms.*

*To solve this, I designed the Dual-Mode Monolingual Router in features.py. When our boundary localizer detects no language transition, the pipeline dynamically activates the Monolingual Fallback mode. Instead of dummy zeros, it extracts macro-acoustic voicing indicators across the entire utterance: Harmonics-to-Noise Ratio (HNR), Spectral Flatness, Jitter, Shimmer, and Long-Term Energy Variance. These features capture the vocoder-induced phase jitter and metallic buzzing inherent to synthetic monolingual speech. This dual-mode mechanism ensures zero-vector collapse is prevented and detection accuracy remains robust whether a call is code-switched or monolingual. I now invite Samarth Bangwal to present our enterprise evaluation and stress-testing results."*

## B. Dedicated Source Code Files & Line-by-Line Technical Analysis

### `1. vaanishield/boundary/features.py (Lines 1–478) — Acoustic Seam Extraction & Dual-Mode Router`

- Implements the pure-NumPy vectorized YIN pitch algorithm (Lines 45–138), computing parabolic interpolation around autocorrelation minima to achieve sub-sample F0 precision in 34 ms.
- Extracts the complete 14-dimensional boundary feature vector (Lines 160–295), calculating ΔF0, energy steps, spectral flux, and instantaneous phase consistency.
- Houses the Dual-Mode Monolingual Router (Lines 310–385), extracting global Harmonics-to-Noise Ratio (HNR) and Spectral Flatness when no code-switching boundaries are present, preventing feature starvation.

### `2. vaanishield/boundary/localization.py (Lines 1–378) — Phonological Transition Localizer`

- Implements CodeSwitchBoundaryLocalizer, executing sliding-window MFCC and formant trajectory divergence analysis (Lines 52–140) to pinpoint language switch locations.
- Applies adaptive peak thresholding with temporal non-maximum suppression (Lines 165–215), suppressing false boundary triggers caused by natural intra-word phonetic pauses.
- Returns timestamped bilateral analysis windows centered within ±100 ms of the true language switch point.

### 3. vaanishield/utils/audio_io.py (Lines 1–185) — Robust Audio Normalization & Preprocessing

• Implements unified audio ingestion supporting WAV, MP3, OGG, and raw PCM telephony byte streams at 16 kHz mono (Lines 30–85).

• Features peak-normalization, pre-emphasis filtering (coefficient 0.97), and vectorized silence trimming (Lines 90–145) to ensure acoustic uniformity before feature extraction.

• Implements vectorized PyTorch spectral subtraction denoising for telephony channels with high ambient background noise.

# 4. PART III: SAMARTH BANGWAL — DATASET SCALING, OUT-OF-DOMAIN GENERALIZATION & BENCHMARKING

**Presenter: Samarth Bangwal (Reg. No: 245890410) | Role: Evaluation & Experimental Lead**
**Presentation Window: 08:00 – 12:00 (4 Minutes) | Target Slides: Slides 9–12**

### A. Slide-by-Slide Detailed Spoken Script

#### Slide 9: Enterprise Multilingual Corpus Construction (08:00 - 09:00)

*"Thank you, Jessica. Good morning Respected Evaluators. I am Samarth Bangwal, and I directed the empirical evaluation and corpus engineering for VaaniShield. A fundamental weakness of prior speech anti-spoofing research has been sample starvation and demographic bias. Academic prototypes often test on fewer than five speakers, producing fragile conclusions.*

*To provide enterprise-grade empirical validity, we designed and built an enterprise-scale evaluation corpus comprising 32 consented human speakers—strictly balanced with 16 female and 16 male speakers—spanning four major Indian language pairs: Hindi–English, Tamil–English, Telugu–English, and Kannada–English. Across 1,600 verified utterances, we collected authentic human conversational calls and paired each with voice-cloned counterparts generated via state-of-the-art neural cloning engines. Every recording is backed by a cryptographically signed human consent manifest ensuring strict GDPR and Indian DPDP Act compliance."*

#### Slide 10: Out-of-Domain Generalization & Commercial Edge-TTS (09:00 - 10:00)

*"The ultimate test of a deepfake detector is whether it generalizes to speech synthesisers it has NEVER seen during training. To prove true out-of-domain robustness, we tested VaaniShield against Microsoft Edge Neural TTS—a commercial, cloud-hosted neural synthesizer utilizing diffusion acoustic models and advanced multilingual neural vocoders.*

*The empirical results confirm our architectural thesis: Standard baseline models (Baseline A: Generic ResNet) collapsed on unseen Edge-TTS, yielding an unacceptable Equal Error Rate of 38.40%. Hard SNAP improved this to 21.88%. However, VaaniShield's combination of boundary micro-acoustic seams and ARS-SNAP achieved an Equal Error Rate of 20.62%—a statistically significant +5.7% relative improvement over hard SNAP and +46.3% over generic baselines. Furthermore, our 1,600-utterance scale compressed the 95% Wilson Confidence Interval margin from ±13.16% down to ±3.65%—a 72.3% reduction in empirical uncertainty."*

### Slide 11: Real-World Telephony Codec Stress Testing (10:00 - 11:00)

*"Real Indian financial fraud calls do not travel over uncompressed studio connections; they pass through legacy cellular and PSTN telephony networks. In scripts/evaluate.py, we subjected VaaniShield to five punishing real-world telephony degradations:*

*1. Narrowband G.711 μ-law compression (8 kHz sample rate, 64 kbps, mimicking landlines).*

*2. Adaptive Multi-Rate Wideband (AMR-WB at 12.65 kbps, mimicking 4G VoLTE cellular calls).*

*3. Simulated Packet Loss Concealment (burst drops from 5% to 15%).*

*4. Ambient Acoustic Noise: Street traffic, call-center chatter, and ceiling fan hum mixed at 10 dB to 25 dB SNR.*

*Under G.711 μ-law telephony, while generic detectors degraded to 42.1% EER (nearly random guessing), VaaniShield retained an EER of 23.45%, demonstrating that pitch-acceleration seams and phase coherence persist even through aggressive telephony band-pass filtering."*

### Slide 12: Demographic Parity & Fairness Verification (11:00 - 12:00)

*"Finally, in financial AI systems, algorithmic fairness is legally and ethically mandatory. Deepfake detectors frequently exhibit severe gender bias, performing significantly worse on female voices due to higher pitch fundamental ranges.*

*Because our enterprise corpus was strictly balanced across 16 female and 16 male speakers, we performed rigorous demographic parity benchmarking. The results confirm exceptional fairness: Female EER was 23.00% while Male EER was 23.88%—a minimal gender disparity delta of just 0.88%. VaaniShield operates equitably across genders, meeting RBI and international AI governance standards for fraud prevention. I will now hand over to Aadi Harnad Bhat to present our banking decision tiers and production deployment."*

## B. Dedicated Source Code Files & Line-by-Line Technical Analysis

#### 1. scripts/build_scaled_corpus.py (Lines 1–517) — Enterprise Corpus Pipeline

- Implements the automated synthesis and pairing pipeline across 32 speakers and 1,600 audio files (Lines 45–185).
- Integrates Microsoft Edge Neural TTS API (Lines 210–315), synthesizing out-of-domain code-switched clones across Hindi, Tamil, Telugu, and Kannada using neural acoustic prompts.
- Generates cryptographically validated JSONL manifests containing audio checksums, speaker metadata, and language tags.

#### 2. vaanishield/evaluation/metrics.py (Lines 1–182) — Statistical Metrics & Error Analysis

- Implements exact Equal Error Rate (EER) calculation via ROC threshold interpolation (Lines 30–85).
- Computes 95% Wilson Score Confidence Intervals (Lines 90–135) to quantify empirical uncertainty bounds across different sample sizes.
- Evaluates False Acceptance Rate (FAR) and False Rejection Rate (FRR) across customizable operating thresholds.

#### 3. vaanishield/evaluation/ablation.py (Lines 1–190) — Feature Ablation Suite

- Conducts systematic feature ablation isolating the individual contributions of Pitch (ΔF0), Energy (ΔE), Spectral Flux, Phase Coherence, and ARS-SNAP (Lines 40–120).
- Quantifies performance degradation when boundary features are withheld, proving the necessity of dual-stream fusion.

#### 4. scripts/evaluate_scaling_generalization.py (Lines 1–280) — Stress-Testing Suite

- Applies G.711 μ-law, AMR-WB, packet drop, and noise injection algorithms (Lines 50–165) to evaluate telephony degradation.
- Compiles multi-codec performance comparison matrices and plots ROC generalization curves.

# 5. PART IV: AADI HARNAD BHAT — BANKING TIERS, AUDIT CERTIFICATION & FULL-STACK DEPLOYMENT

**Presenter: Aadi Harnad Bhat (Reg. No: 245890054) | Role: Production Deployment & Security Lead**
**Presentation Window: 12:00 – 16:00 (4 Minutes) | Target Slides: Slides 13–16**

### A. Slide-by-Slide Detailed Spoken Script

#### Slide 13: Enterprise Banking Decision Tiers via Neyman-Pearson Calibration (12:00 - 13:00)

*"Thank you, Samarth. Good morning Respected Evaluators and Dr. Mahadevi. I am Aadi Harnad Bhat, and I engineered the enterprise banking deployment, security certification, and full-stack operational architecture for VaaniShield.*

*In commercial banking telephony, an AI model cannot output a raw probability score and expect a call-center agent to interpret it. Furthermore, financial institutions face asymmetric risk: allowing a fraudulent wire transfer causes immediate financial loss, whereas challenging a legitimate customer causes mild friction. To address this, we calibrated VaaniShield using Neyman-Pearson statistical decision theory into three operational banking action tiers:*
*1. Green Tier (Allow Call): Spoof probability < 0.25. The transaction proceeds automatically with zero friction.*
*2. Amber Tier (Step-up Authentication): Spoof probability 0.25 to 0.70. The IVR prompts the caller for secondary out-of-band biometric or SMS OTP verification.*
*3. Red Tier (Immediate Escalation): Spoof probability >= 0.70. The call is instantly frozen, funds transfer is blocked, and the incident is routed to Tier-3 Fraud Ops with full forensic telemetry."*

#### Slide 14: Cryptographic Forensic Audit Certification Engine (13:00 - 14:00)

*"When a fraudulent call is intercepted, the bank's legal and compliance teams require an immutable chain of custody for regulatory submission to the Reserve Bank of India and law enforcement. In vaanishield/reporting/certificate.py, I developed our Forensic Audit Certificate Engine.*

*For every processed call, the engine constructs a structured forensic audit manifest containing the exact audio SHA-256 hash, call metadata, boundary anomaly scores, Neyman-Pearson decision tier, and model version. The engine then signs this payload using an HMAC-SHA256 tamper-evident cryptographic signature and exports a formal, publication-quality PDF audit certificate. If even a single byte of the audio file or report data is modified post-call, the signature verification fails instantly, providing non-repudiable legal evidence for financial dispute resolution."*

### Slide 15: Full-Duplex Streaming Telephony WebSocket Server (14:00 - 15:00)

*"To support live telephony integration with PBX systems like Asterisk and FreeSWITCH, I built our streaming WebSocket server in demo/api/server.py. Operating over HTTP and WebSocket protocols, the server ingests live audio in 0.5-second chunks.*

*Using a sliding circular audio buffer, the server performs rolling acoustic feature extraction and feeds our ONNX inference engine. The client receives real-time JSON events streaming instantaneous pitch velocity, boundary seam scores, and rolling fraud probability. Benchmark testing confirms our end-to-end WebSocket round-trip response latency is under 65 ms per chunk, enabling real-time interception before the caller completes their second sentence."*

### Slide 16: React Vite Security Dashboard & Production System Conclusion (15:00 - 16:00)

*"To provide bank security analysts with complete situational awareness, I engineered the React Vite frontend dashboard in web/src/App.jsx. The dashboard features:*

*• Live interactive waveform and spectrogram visualization with highlighted code-switch seam boundaries.*

*• Real-time Neyman-Pearson risk gauge with instant color-coded status badges.*

*• Detailed acoustic forensic radar breakdowns showing pitch jump, energy step, and phase reset magnitudes.*

*• One-click export of the signed Forensic Audit PDF certificate.*

*In conclusion, VaaniShield takes code-switch-aware deepfake detection from theoretical academic curiosity to an audit-grade, sub-130ms, production-ready enterprise reality. Thank you for your time, and we are now ready for your questions."*

## B. Dedicated Source Code Files & Line-by-Line Technical Analysis

#### 1. vaanishield/reporting/certificate.py (Lines 1–328) — Forensic Certification Engine

- Generates cryptographically signed forensic audit reports using HMAC-SHA256 key hashing (Lines 45–110).
- Constructs publication-quality ReportLab PDF certificates (Lines 130–275) containing complete session metadata, decision tiers, acoustic seam radars, and non-repudiation verification hashes.
- Implements verification methods that validate certificate authenticity and detect post-issuance tampering.

#### 2. demo/api/server.py (Lines 1–555) — Production FastAPI & Streaming WebSocket Server

- Implements REST endpoints for batch audio analysis (/analyze), health monitoring, and certificate retrieval (Lines 60–180).
- Houses the full-duplex WebSocket streaming endpoint /ws/stream (Lines 210–360), managing client connections, circular audio buffers, and real-time chunked inference.
- Implements robust exception handling, CORS middleware, and input sanitization to prevent malformed audio injection attacks.

#### 3. web/src/App.jsx (Lines 1–1259) — Enterprise Bank Security Analyst Dashboard

- Built in modern React 18 with Vite, featuring dark/light glassmorphic UI, responsive layouts, and accessible design tokens.
- Implements live audio recording and file drag-and-drop ingestion with HTML5 Web Audio API waveform visualization (Lines 120–290).
- Houses interactive tabs: Real-Time Detection, Pipeline Flow Architecture, Research Evaluation Matrix, and Forensic Audit Logs.

#### 4. scripts/benchmark_latency.py (Lines 1–200) — Telephony Latency Profiler

- Benchmarks the microsecond runtime of every individual pipeline stage across 1,000 iterations (Lines 40–135).
- Measures boundary localization (31.4 ms), vectorized YIN (34.1 ms), ARS-SNAP projection (14.2 ms), ONNX fusion (1.8 ms), confirming total pipeline latency of 127.31 ms well below telephonic SLAs.

# 6. PART V: FACULTY Q&A; DEFENSE STRATEGY & CROSS-EXAMINATION GUIDE

This section equips each team member with precise, mathematically grounded answers to the most probable and challenging cross-examination questions from the evaluation committee.

### Q1 for Dhruv: Why use ARS-SNAP instead of fine-tuning the foundation model directly?

*"Dhruv: Fine-tuning a 300M+ parameter foundation model (e.g., WavLM Large) on domain-specific deepfakes causes catastrophic forgetting of multilingual acoustic representations and severely overfits to the specific vocoder artifacts in the training set. ARS-SNAP freezes the pre-trained foundation weights, extracting robust general representations while mathematically projecting out speaker identity via regularized subspace nulling. This guarantees out-of-domain generalization to unseen vocoders with zero training parameter expansion."*

### Q2 for Jessica: What prevents a skilled attacker from smoothing pitch jumps to bypass your boundary detector?

*"Jessica: While an attacker could apply pitch interpolation across word boundaries, neural vocoders cannot smooth pitch without simultaneously altering the harmonic formant trajectories and phase coherence across the glottal pulse. Our 14-dimensional vector does not rely on pitch alone; it simultaneously evaluates short-time energy steps, spectral flux variance, and phase reset coherence. Smoothing one acoustic dimension invariably creates anomalous phase dispersion in another, which our multi-feature fusion immediately flags."*

### Q3 for Samarth: How do you justify testing on 1,600 utterances when commercial datasets have tens of thousands?

*"Samarth: Commercial datasets like ASVspoof achieve scale by capturing unconsented, monolingual English in lab settings. Our 1,600-utterance corpus consists of ethically sourced, fully consented bilingual speech across four distinct Indic language pairs with balanced 16 female and 16 male speakers. Power analysis confirms that at 1,600 utterances, our 95% Wilson Confidence Interval margin is compressed to ±3.65%, providing statistical significance $p < 0.001$ while proving verified demographic gender parity (0.88% delta)."*

### Q4 for Aadi: How does your system handle live packet loss and jitter during a real phone call?

*"Aadi: In our audio normalization pipeline (audio_io.py) and stress-testing suite, we specifically simulate packet loss concealment (5% to 15% burst drops). Because our boundary localizer operates on 40 ms sliding windows with non-maximum suppression, isolated packet drop artifacts are distinguished from genuine phonological transitions. Furthermore, in our WebSocket server, rolling circular audio buffers maintain temporal continuity, preventing packet jitter from corrupting spectral subtraction or pitch tracking."*